Introduction to Artificial Intelligence

Semester I, 2026-27

Problem Solving with Local Search

Rohan Paul

Outline

- Last Class
  - Constraint Satisfaction Problems
- This Class
  - Local Search Algorithms
- Reference Material
  - AIMA Ch. 4.1

Acknowledgement

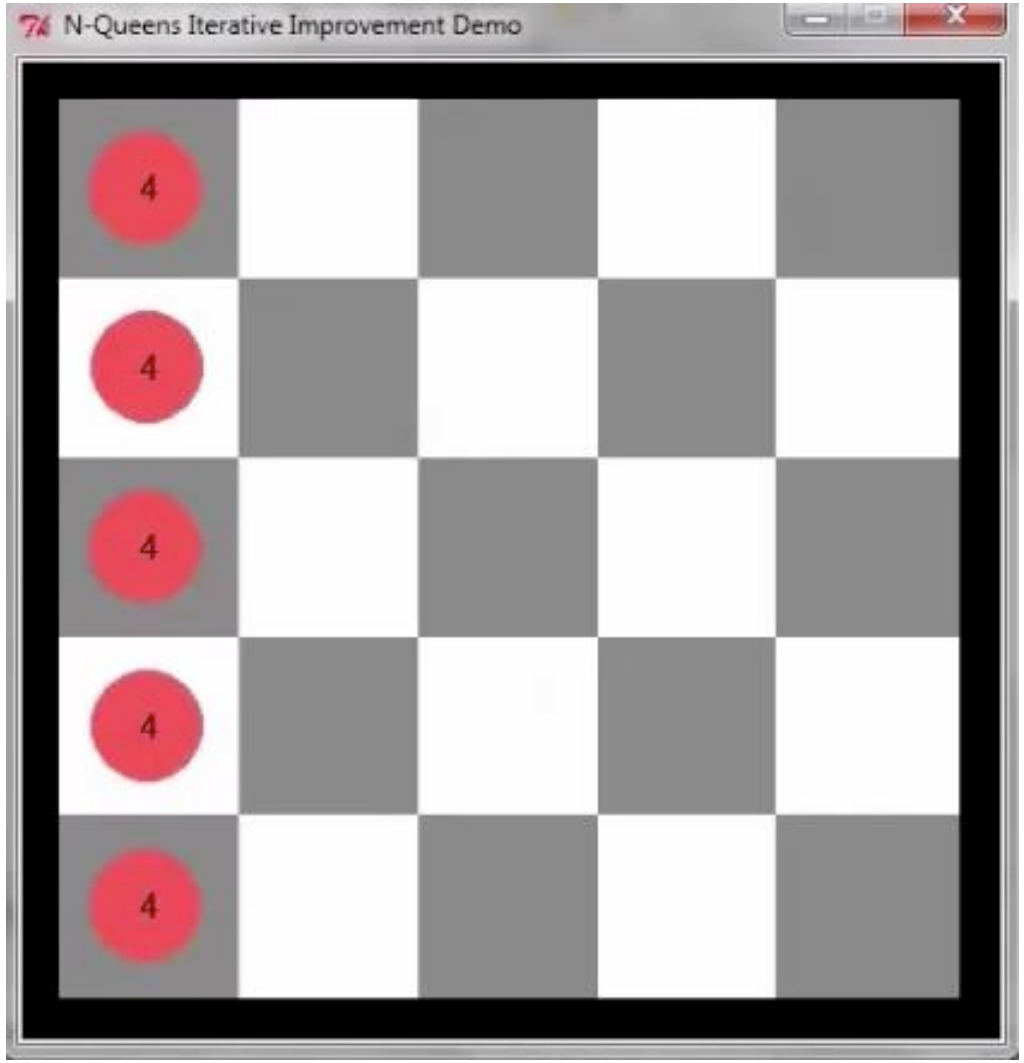
These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Doina Precup, Dorsa Sadigh, Percy Liang, Mausam, Dan Klein, Nicholas Roy and others.

Another view of constraint satisfaction

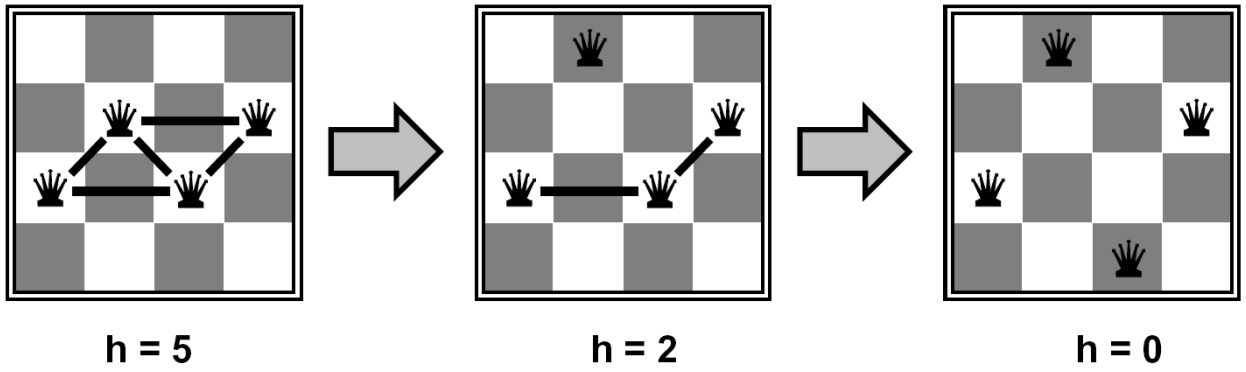
- Constraint Satisfaction Objective
  - To find a solution (a set of variable to value assignments) that satisfy a constraint.
- Can we view constraint satisfaction as **an iterative "search"?**
  - With a cost (evaluation function)
    - The degree of unsatisfaction (# of violated constraints)
  - Solution search is an optimization process that tries to minimize this "cost", i.e., reducing that number of constraints violated.
  - The "optimal" solution with zero cost is the solution that does not violate any constraint.

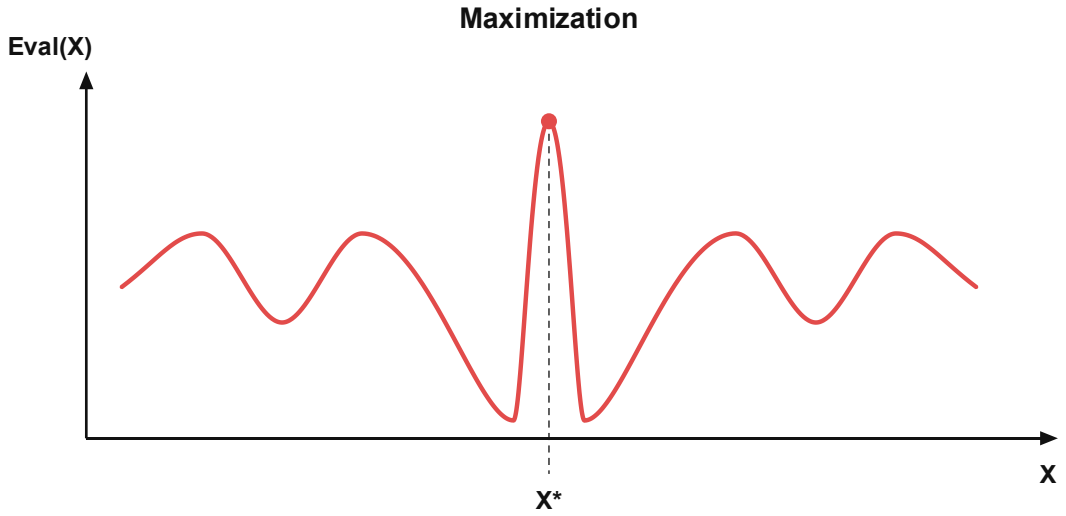
Another view of CSPs

Idea:
Start with an imperfect solution and then iteratively improve from the present state.



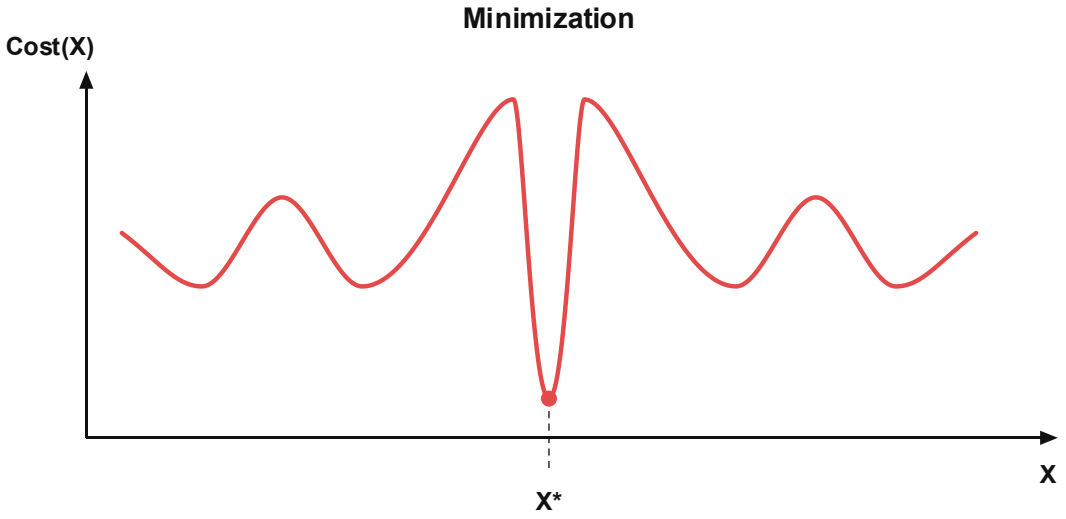
CSP solving with (local) improvements





Solving CSPs iteratively

- Think in an iterative manner. We start with some candidate assignment (solution), X.
- It may have a cost or evaluation.
- We want to iteratively “decide” moves that reduce cost or improve evaluation.

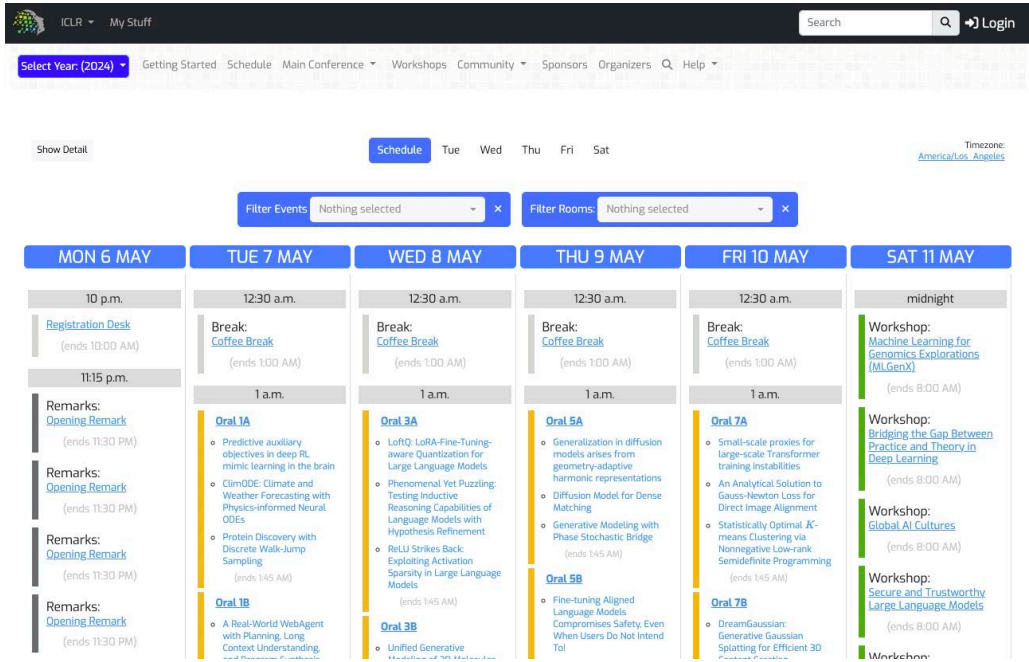


Another example: Conference Scheduling

ICRA 2024 Technical Program Tuesday May 14, 2024 Tuesday Wednesday Thursday Next Top

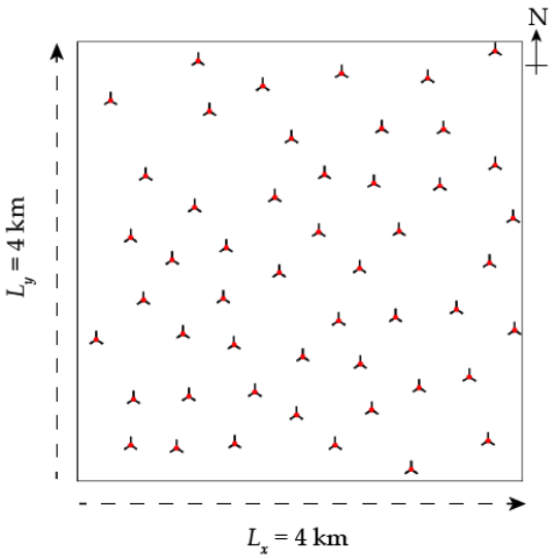
Click on the day's program and use the arrow keys for easy horizontal and vertical scrolling

Keynote 1	Award Session 1	Keynote 2	Award Session 2	Keynote 3	Technical Session 1	Technical Session 2	Technical Session 3	Technical Session 4	Technical Session 5	Technical Session 6	Te
	10:30-12:00 CC-Main Hall Award Session TuAA1-CC *Automation*		10:30-12:00 CC-301 Award Session TuAA2-CC *Cognitive Robotics*		10:30-12:00 CC-303 Oral Session TuAT1-CC *Planning under Uncertainty I*	10:30-12:00 CC-311 Oral Session TuAT2-CC *Mechanism Design I*	10:30-12:00 CC-313 Oral Session TuAT3-CC *Formal Methods in Robotics and Automation I*	10:30-12:00 CC-315 Oral Session TuAT4-CC *Multi-Robot Systems I*	10:30-12:00 CC-411 Oral Session TuAT5-CC *Sensors and Audition*	10:30-12:00 CC-414 Oral Session TuAT6-CC *2D/3D Visual Perception*	
	13:30-15:00 CC-Main Hall Award Session TuBA1-CC *Human-Robot Interaction*		13:30-15:00 CC-301 Award Session TuBA2-CC *Mechanisms and Design*		13:30-15:00 CC-303 Oral Session TuBT1-CC *Planning under Uncertainty II*	13:30-15:00 CC-311 Oral Session TuBT2-CC *Mechanism Design II*	13:30-15:00 CC-313 Oral Session TuBT3-CC *Formal Methods in Robotics and Automation II*	13:30-15:00 CC-315 Oral Session TuBT4-CC *Multi-Robot Systems II*	13:30-15:00 CC-411 Oral Session TuBT5-CC *Vision Systems*	13:30-15:00 CC-414 Oral Session TuBT6-CC *RGB-D Sensing and Perception I*	In
15:30-16:00 National Convention Hall Keynote Session TuKN1-HI		15:30-16:00 CC-Main Hall Keynote Session TuKN2-CC		15:30-16:00 CC-301 Keynote Session TuKN3-CC							



Assign papers that are similar in a session. Avoid conflicts between sessions. This is a complex CSP problem.

Local search is a tool for general discrete optimization problems



Windmill Placement Problem: Optimizing the locations of windmills in a wind farm

- An area to place windmills. Location of windmills affects the others. Reduced efficiency for those in the wake of others.
- Grid the area into bins. A large number of configurations of windmills possible.
- Given a configuration we can evaluate the total efficiency of the farm.
- Can neither enumerate all configurations nor optimize the power efficiency function analytically.
- **Goal is to search for the configuration that maximizes the efficiency.**

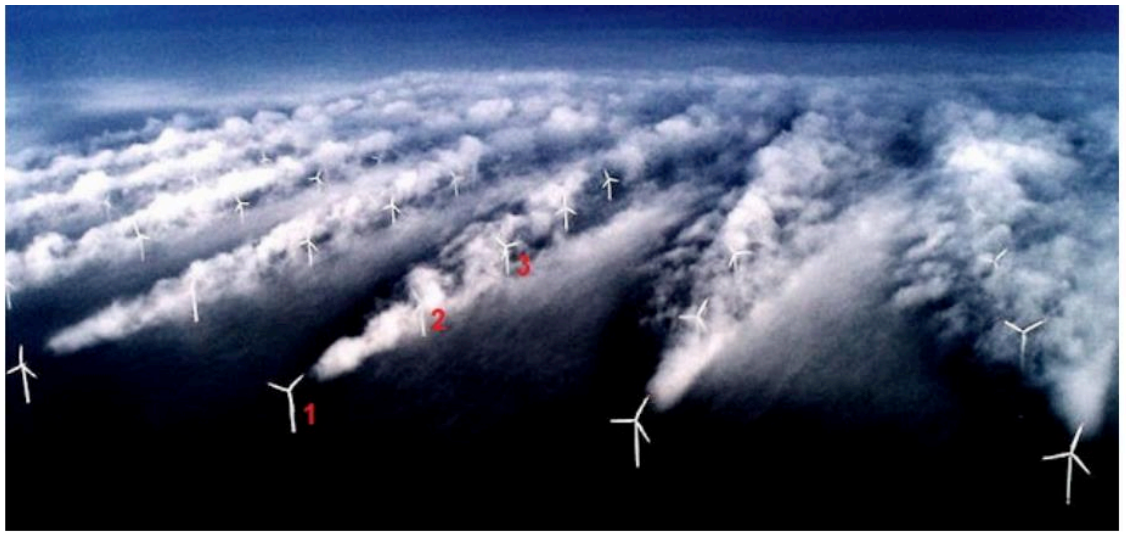


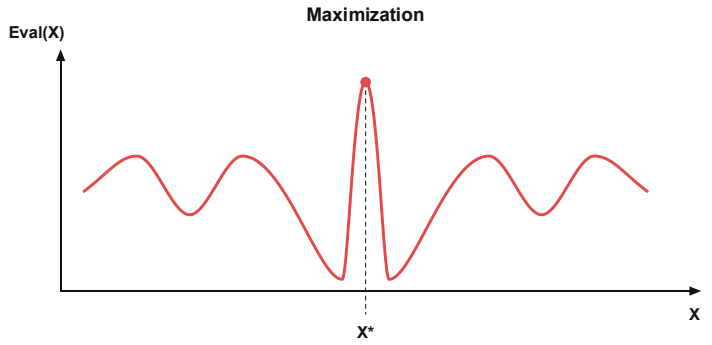
Figure 5: Turbines experiencing multiple wakes. As an example, turbine 3 is experiencing wake effects from both turbine 1 and 2. Image adopted from [4].

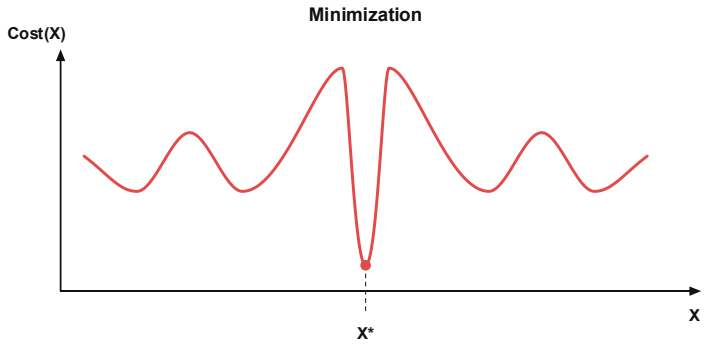
Inspired from this example: https://www.shell.in/energy-and-innovation/ai-hackathon/_jcr_content/par/textimage_1834506119_1619963074.stream/1612943059963/4b0a86b7cc0fe7179148284ffed9ef33524c2816/windfarm-layout-optimisation-challenge.pdf

General Recipe for Solving Discrete Optimization Problems

- **Setting**
  - A set of discrete states, *X*.
  - An objective/evaluation function assigns a "goodness" value to a state, *Eval(X)*
  - Problem is to search the state space for the state, *X** that maximizes the objective.

- **Searching for the optimal solution can be challenging. Why?**
  - The number of states is very large.
    - Cannot simply enumerate all states and find the optimal.
    - The path to get to the optimal solution is not relevant.
  - We can only evaluate the function.
    - Cannot write it down analytically and optimize it directly.





- Searching for "the optimal" solution is very difficult.
- Question is whether we can search for a reasonably good solution.

Local Search Methods

- **Keep track of a single "current" state**
  - We need a principled way to search/explore the state space hoping to find the state with the optimal evaluation.
  - Do not maintain a search tree as we need the solution not the path that led to the solution.
  - Only maintain a single current state.

- **Perform local improvements**
  - Look for alternatives in the vicinity of that solution
  - Try to move towards more better solutions.

A simple local search approach: *Hill-climbing Search*

Hill climbing



Let S be the start node and let G be the goal node.

Let h(c) be a heuristic function giving the value of a node

Let c be the start node

Loop

Let c' = the highest valued neighbor of c

If h(c) ≥ h(c') then return c

c = c'

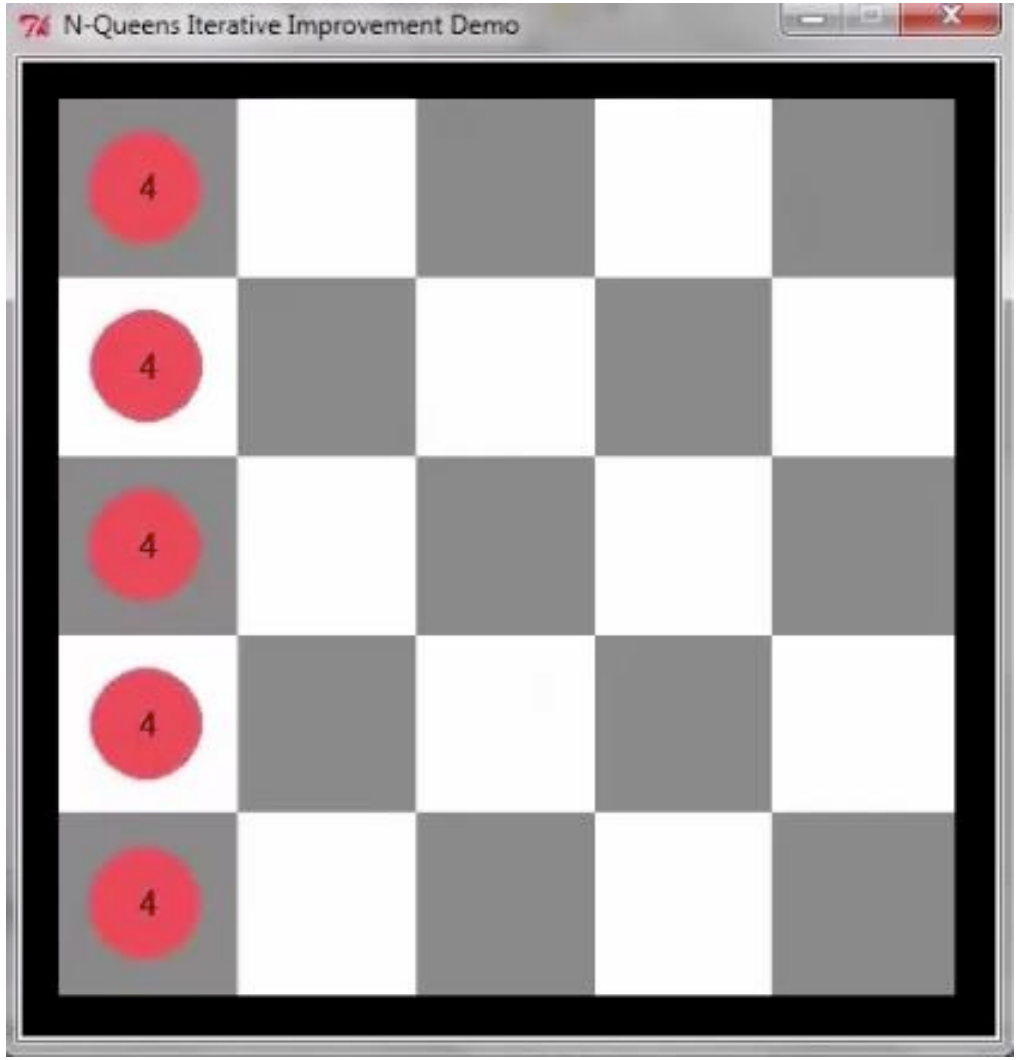
Start at a configuration. Evaluate the neighbors. Move to the highest valued neighbor if its value is higher than the current state. Else stay.

"Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node
   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
   end
```

Hill climbing for 4 -queens

- Select a column and move the queen to the square with the fewest conflicts.
- Perform local modifications to the state by changing the position of one piece till the evaluation is minimum.
- Evaluate the possibilities from a state and then jump to that state.



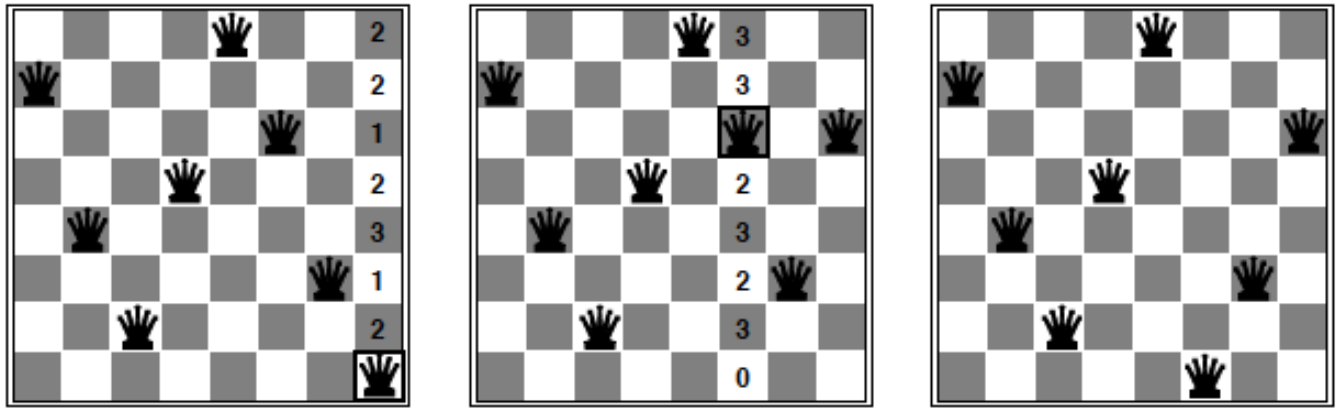
Example

- Local search looks at a state and its local neighborhood.
- Not constructing the entire search tree.
- Consider local modifications to the state. Immediately jump to the next promising neighbor state. Then start again.
- *Highly scalable.*

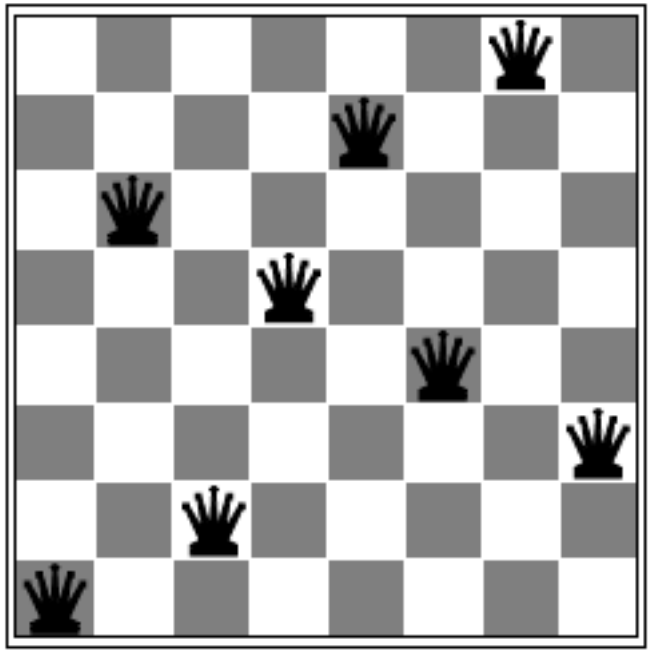


Susceptibility to Local Minima

Limitation: Search reaches a solution where it cannot be improved - a local minimum.

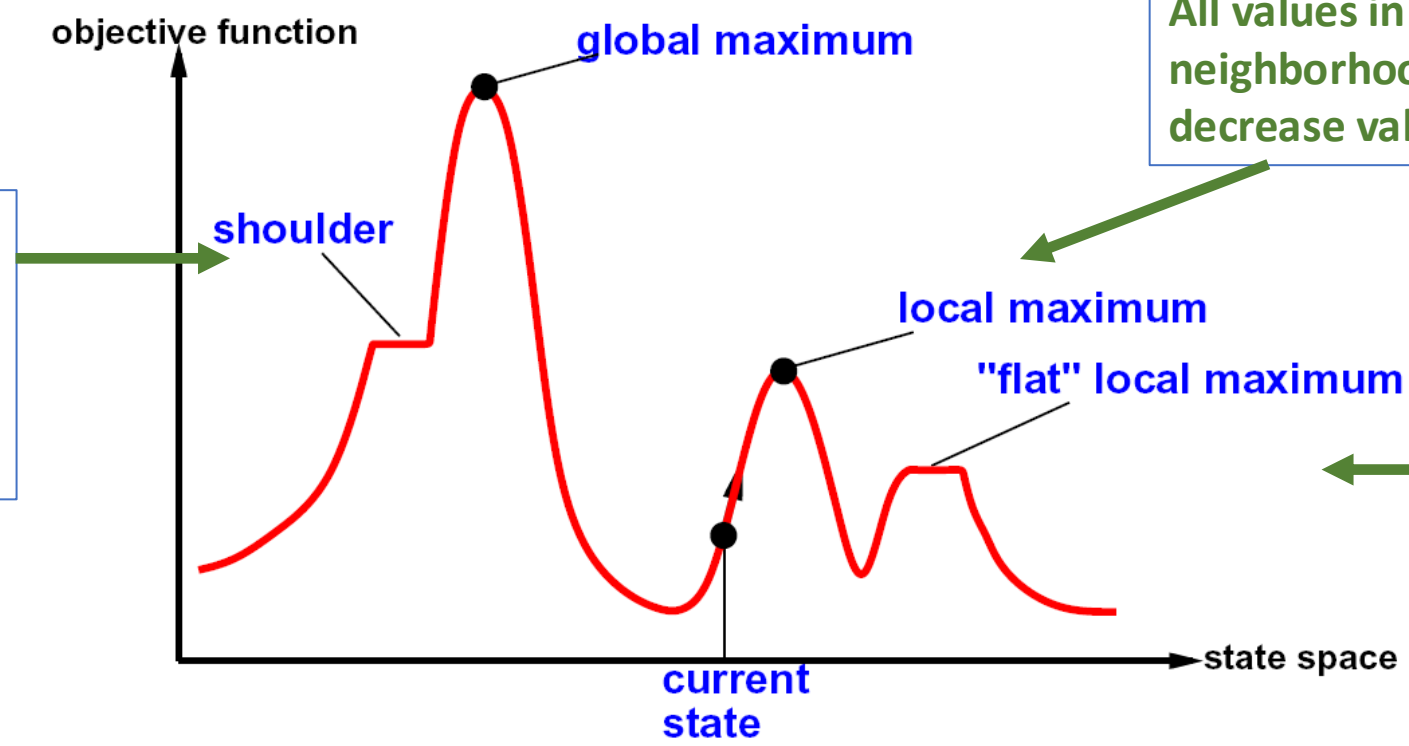


Is this an optimal state?



Local minima (h = 1). Every successor has a higher cost.

Core Problem in Local Search



All values in the neighborhood decrease values

Neighborhood values are equivalent, need to search for a longer distance to find a path to a higher value

Neighborhood values are equivalent beyond which values only decrease

- Hill climbing is prone to local maxima. Neighbors may not be of higher value. Search will stop at a sub-optimal solution
- **Locally optimal actions may not lead to the globally optimal solution.**

Escaping Local Minima

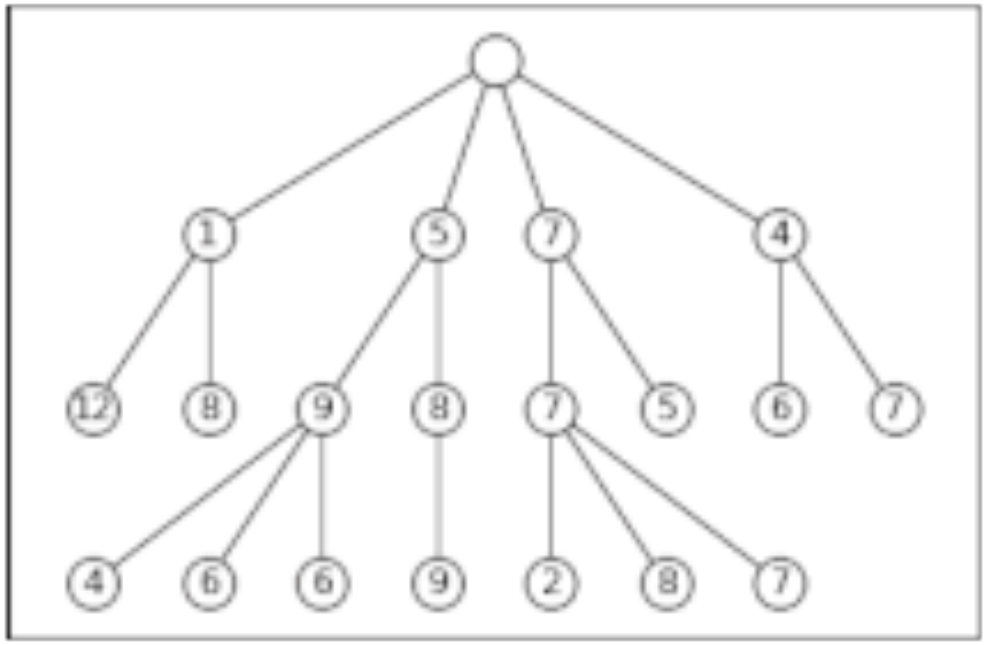
Looking for Solution from Multiple Points

- **Local Beam Search**
- Algorithm
  - Track k states (rather than 1).
  - Begin with k randomly sampled states.
  - Loop
    - Generate successors of each of the k-states
    - If anyone has the goal, the algorithm halts
    - Otherwise, select only the k-best successors from the list and repeat.

- Note:
  - Each run is not independent, information is passed between parallel search threads.
  - Promising states are propagated. Less promising states are not propagated.
  - Problem: states become concentrated in a small region of space.

Note: Beam Search is a General Search Technique

- Beam search is a general idea (see right figure).
- Instead of considering all solutions at a level, consider only the top-k.
- Note: usually our memory is finite in size, there is an upper bound on the number of states that can be kept.
- In general, it is an approximate search method.



Beam search is a general idea. Here, shown in the context of a tree search. Beam size is 3. For local search we don't construct the full tree.

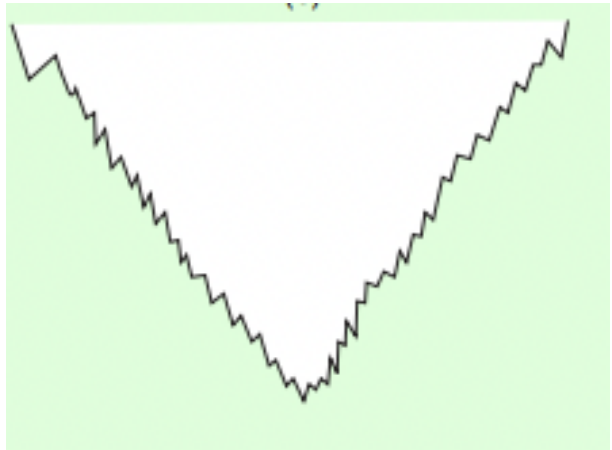
Disadvantage: in the context of local search, the search may get concentrated in a small region of space (in which case it becomes like hill climbing.)

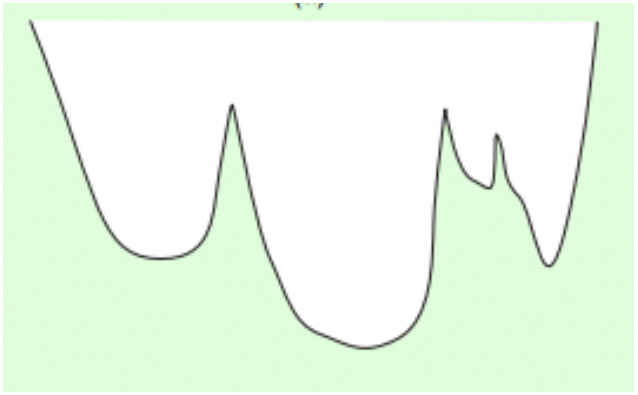
Source: wikipedia

Escaping Local Minima
Adding Randomness

- **Random Re-starts**
  - A series of searches from randomly generated initial states.
- **Random Walk**
  - Pick ”any” candidate move (whether improves the solution or not).
- ***Stochastic* Hill Climbing**
  - Instead of picking the *best move, p*ick *any* move that produces an *improvement*.
- ***Stochastic* beam search**
  - Instead of taking the best k states
  - Sample k states from a distribution
  - Probability of selecting a state *increases* as the *value* of the state.

Q: Which method to use for the following cost surfaces? Random re-starts or random walk?

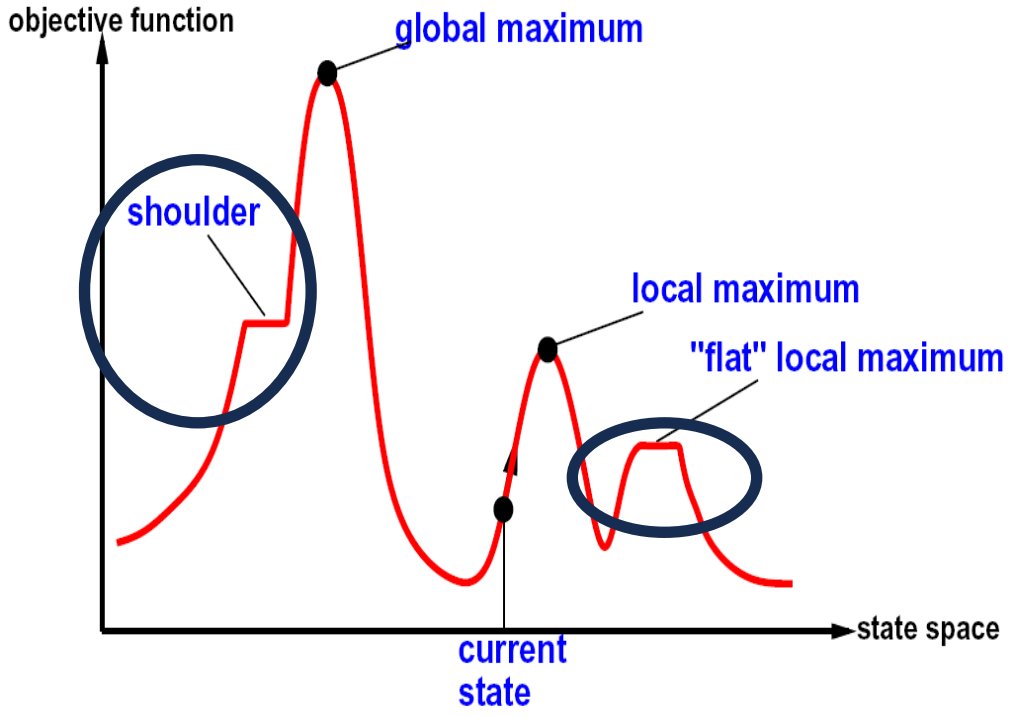




Escaping Local Minima

Explore the local states a few steps.

- **Keep Moving**
  - Make sideways moves for a few steps.
  - Useful way to escape flat local minima (shoulders)
  - When local search reaches a flat area, that is, when all the neighbours have the same cost as the current state, it terminates right away



Key Takeaways

- Problem characteristic
  - Looking for an optimal solution.
  - Can only evaluate the cost function/evaluation function – it is not available analytically to us – can’t optimize it analytically.
- Solution Approach
  - Start in some state and locally improve the solution.
  - It is an intuitive way in which we solve such complex problems.
  - Local search approach: Hill climbing.
- Challenge
  - Local minima. The local nature of the algorithm may lead to solutions that may not be globally optimal.